Hack Todo List に必要なデータベース

結論

- 必須DBは **Supabase PostgreSQL**（1つ）で十分。
- 認証は Supabase Auth、業務データは PostgreSQL テーブルで管理する。

必須テーブル一覧

1. profiles

- 目的: ユーザーの追加情報と設定管理
- 役割: Supabase Auth の `auth.users` をアプリ側で補完するプロフィール基盤
- 説明: ログイン情報そのものは `auth.users` にあり、表示名・タイムゾーンなどアプリ独自情報をこのテーブルで保持する。
- 主なカラム:
  - id (uuid, PK, auth.users.id と同一)
  - display_name (text)
  - timezone (text)
  - created_at (timestamptz)

2. todos

- 目的: Todo管理
- 役割: 日々のタスクを保存し、完了状況や優先度を管理する中核データ
- 説明: フロントエンドのTodo一覧・進捗表示の元データ。 `due_date` と `status` を使って未完了タスクや期限超過を判定する。
- 主なカラム:
  - id (uuid, PK)
  - user_id (uuid, FK -> profiles.id)
  - title (text)
  - description (text)
  - due_date (date)
  - status (text: todo / doing / done)
  - priority (smallint)

    - created_at (timestamptz)
    - updated_at (timestamptz)

3. calendar_events

- 目的: カレンダー機能・日付管理
- 役割: 予定や締切を時系列で可視化するカレンダーのイベントデータ
- 説明: 予定の開始/終了時刻を保持し、カレンダーUIや日別表示のデータソースとして利用する。 `all_day` で終日イベントにも対応する。
- 主なカラム:
    - id (uuid, PK)
    - user_id (uuid, FK -> profiles.id)
    - title (text)
    - start_at (timestamptz)
    - end_at (timestamptz)
    - all_day (boolean)
    - created_at (timestamptz)

4. tab_sessions

- 目的: タブ滞在ログの保存（拡張機能から送信）
- 役割: 「どのURLに何秒いたか」を記録する行動ログの事実テーブル
- 説明: Chrome拡張機能から送られる滞在情報を時系列保存し、浪費時間グラフ・警告判定・金額換算の一次データとして使う。
- 主なカラム:
    - id (bigserial, PK)
    - user_id (uuid, FK -> profiles.id)
    - domain (text)
    - url (text)
    - tab_title (text)
    - started_at (timestamptz)
    - ended_at (timestamptz)
    - duration_sec (integer)
    - is_waste (boolean)
    - created_at (timestamptz)

5. alert_rules

- 目的: 「一定時間超過で巨大警告」の条件管理

- 役割: ドメインごとの警告ポリシー（しきい値）を管理する設定テーブル
- 説明: 例として `youtube.com` を 900 秒に設定すると、滞在がしきい値を超えた時に拡張機能側でオーバーレイ警告を出す。
- 主なカラム:
  - id (uuid, PK)
  - user_id (uuid, FK -> profiles.id)
  - target_domain (text)
  - threshold_sec (integer)
  - enabled (boolean)
  - created_at (timestamptz)

6. btc_price_history

- 目的: BTC価格履歴（換算の再現性確保）
- 役割: 時点ごとのBTC価格を保存する参照データ
- 説明: 「その時点の価格で換算した結果」を後から再計算・検証できるようにする。外部API障害時のキャッシュとしても機能する。
- 主なカラム:
  - id (bigserial, PK)
  - source (text)
  - price_jpy (numeric)
  - fetched_at (timestamptz)

7. waste_cost_snapshots

- 目的: 浪費時間を金額換算した集計結果
- 役割: グラフ表示向けの集計済みKPIデータ
- 説明: 生ログ (`tab_sessions`) から計算した「浪費秒数」「換算金額」を期間単位で保持し、ダッシュボードを高速表示する。
- 主なカラム:
  - id (bigserial, PK)
  - user_id (uuid, FK -> profiles.id)
  - period_start (timestamptz)
  - period_end (timestamptz)
  - wasted_seconds (integer)
  - btc_price_jpy (numeric)
  - lost_amount_jpy (numeric)
  - created_at (timestamptz)

インデックス（最低限）

- tab_sessions(user_id, started_at)
- todos(user_id, due_date)
- calendar_events(user_id, start_at)
- waste_cost_snapshots(user_id, period_start)

セキュリティ

- Supabase RLS を有効化する。
- すべてのユーザーデータ系テーブルは「自分の user_id のみ SELECT/INSERT/UPDATE/DELETE 可」にする。

補足（任意）

- 将来データ量が増えたら tab_sessions を月単位でパーティション化。
- 高速通知やレート制御が必要なら Redis を追加検討。